

Arquitetura que Escala

Capítulo de amostra Camadas, domínio e fronteiras que sobrevivem ao crescimento

por Carlos Ferreira

Material de estudo da plataforma MentorHub mentorhub.demo

1. Por que arquitetura importa (mesmo no começo)

Todo sistema nasce simples. Um par de pastas, um arquivo de rotas, um banco recém-criado. E é exatamente aí que a arquitetura decide o futuro do projeto não quando ele "fica grande", mas muito antes.

Arquitetura não é sobre diagramas bonitos nem sobre escolher o framework da moda. É sobre **onde as decisões vivem** e **o quão caro é mudar de ideia**. Um sistema bem arquitetado é aquele em que trocar o banco, mudar uma regra de negócio ou redesenhar uma tela toca poucos arquivos porque as fronteiras já existem.

O custo invisível da mescla

Quando a UI fala direto com o banco, cada tela vira um ponto de decisão duplicado. Regras de desconto aparecem em três lugares. Formatação de moeda tem duas implementações. Nenhum desses problemas aparece no demo aparecem no terceiro mês, quando o time dobra e o backlog não anda.

A conta é simples: arquitetura é o que faz o custo de UMA mudança ser $O(1)$ em vez de $O(N)$. E o N , em software, sempre cresce.

2. As quatro camadas que sobrevivem ao tempo

Depois de 12 anos refatorando sistemas alheios, cheguei a um conjunto mínimo de camadas que funciona para quase tudo:

UI (interface)

Componentes, telas, estados visuais. A camada mais "burra" do sistema e é elogio: ela só exibe e coleta. Nada de regra de negócio, nada de SQL.

Aplicação (casos de uso)

Os verbos do sistema: InscreverAluno, PublicarCurso, CancelarAgendamento. Cada caso de uso valida entrada, aplica regras, executa efeitos e devolve um resultado claro. É a camada mais importante e a mais esquecida.

Domínio

As regras que fazem o SEU negócio ser o seu negócio: como se calcula preço, quando um pedido pode ser cancelado, o que significa "matrícula ativa". Puro, testável, sem framework.

Infraestrutura

HTTP, banco, filas, cache, e-mail. Detalhe plugável nunca protagonista.

3. A regra de ouro das dependências

Se você levar uma única frase deste capítulo, que seja esta:

****Dependências apontam para dentro.****

UI conhece Aplicação. Aplicação conhece Domínio. Infraestrutura é plugada na Aplicação nunca o contrário. O domínio não importa React, nem o SDK do banco, nem o cliente HTTP.

Como testar se seu projeto respeita a regra

Abra a pasta do domínio e procure por imports de bibliotecas de UI ou de banco. Achou? Você não tem domínio tem uma bagunça com nome bonito.

O teste prático: "consigo rodar minhas regras de negócio num script de terminal, sem navegador, sem servidor?" Se sim, suas fronteiras estão de pé.

4. Casos de uso: os verbos do sistema

Um caso de uso bem escrito é uma intenção com nome de negócio. Não "handler", não "controller" **UseCase**.

Anatomia mínima

1. **Entrada tipada**: o que o mundo exterior precisa fornecer (dados validados).
2. **Autorização**: quem pode fazer isso?
3. **Regras**: o que o negócio exige para permitir?
4. **Efeitos**: persistir, notificar, disparar eventos.
5. **Resultado**: sucesso com dados, ou erro de negócio nomeado.

O erro mais comum

Colocar a regra no lugar errado. "Aluno não pode se inscrever em curso despublicado" é REGRA (domínio/aplicação), não um if na tela. Regra espalhada em if de UI é dívida garantida: ela vai ser esquecida na próxima tela que fizer a mesma operação.

5. Refatoração incremental: o método das duas semanas

Ninguém reescreve um sistema em um fim de semana e quem tenta, geralmente entrega dois sistemas ruins. O método que uso com mentorados:

Semana 1 Mapear

Liste as 10 operações de negócio mais frequentes do sistema. Para cada uma, anote em quantos arquivos a regra aparece hoje. Esse número é seu baseline.

Semana 2 Centralizar

Escolha as 3 regras mais duplicadas e centralize cada uma em um módulo próprio (use case). Atualize os chamadores. Meça de novo.

A diferença entre os dois números é a sua primeira vitória mensurável de arquitetura e o argumento técnico mais forte que você pode levar para a próxima discussão de prioridade com o time.

Onde continuar

Este capítulo é uma amostra do curso ****Arquitetura de Software na Prática****, onde refatoramos um sistema real aula por aula com checklists, revisão de código e mentoria 1:1 incluída.

Checklist final do capítulo

- Existe pasta/módulo de domínio sem imports de framework?
- Cada operação de negócio tem UM caso de uso com nome claro?
- A UI chama casos de uso nunca banco/HTTP direto?
- Você consegue explicar as camadas em 30 segundos para um dev novo?

Se marcou quatro "sim", seu projeto está pronto para escalar. Se marcou menos de dois, comece pela regra de ouro das dependências é a alavanca mais barata que existe.